

Software Testing

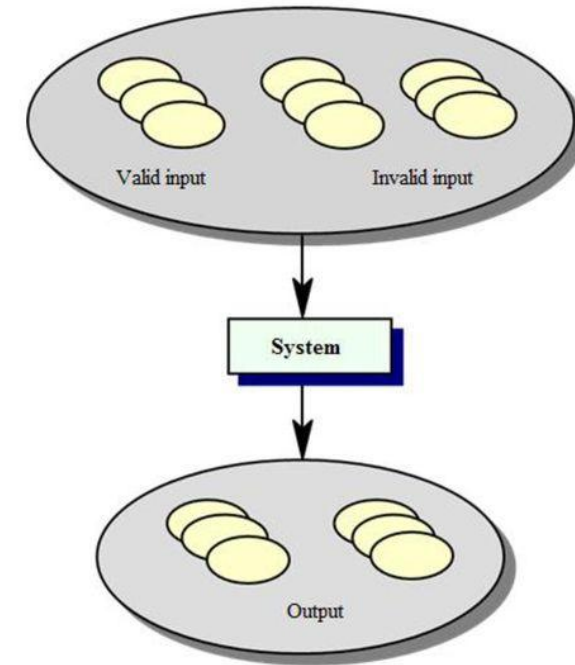
Course's Code: CSE 453

Test Design Techniques – Equivalent Class Partition
(ECP)

Equivalence partitioning

- Divides the input data of a software unit into partitions of equivalent data (**groups of similar data**) from which test cases can be derived
- reduce the number of test cases
- Also Known as Equivalence Class Partitioning

Equivalence partitioning graphically



Equivalence partitioning

What can be partitioned?



- Usually input data is partitioned.
- In some rare cases output data can be partitioned as well.
- Partition represents a set of valid or invalid states for input conditions. An input has certain ranges which are valid and other ranges which are invalid.
- Each partition is expected to be treated the same way by the software.

Equivalence partitioning



How it works?

- Take each input's or output's condition that is described in the specification and derive at least 2 classes for it:
 - One class that satisfies the condition – the **valid class**.
 - Second class that doesn't satisfy the condition – the **invalid class**.
- Design test cases based on the equivalence classes.

Equivalence partitioning



Example

- A program which accepts an integer in the range **1** to **10**.
 - Three partitions generated:
 - $x < 1$;
 - $1 \leq x \leq 10$;
 - $x > 10$.
 - Test cases (One value from each partition):
 - $x = -1$;
 - $x = 5$;
 - $x = 12$.

Equivalence partitioning

Tips for partitioning



- If an input condition specifies a range, at the minimum one valid and two invalid partitions can be defined.
- If an input condition requires a specific value, then one valid and one invalid partition can be defined.
- If an input condition specifies a member of a set, then one valid and one invalid partition can be defined.
- If an input condition is Boolean, then one valid and one invalid partition can be defined.

Examples on Equivalence partitioning

Let's check our requirement: we need to develop a program that tells us whether a year is a leap year or not.

A leap year is every year that is exactly divisible by four is a leap year, except for years that are exactly divisible by 100, but these centurial years are leap years if they are exactly divisible by 400.

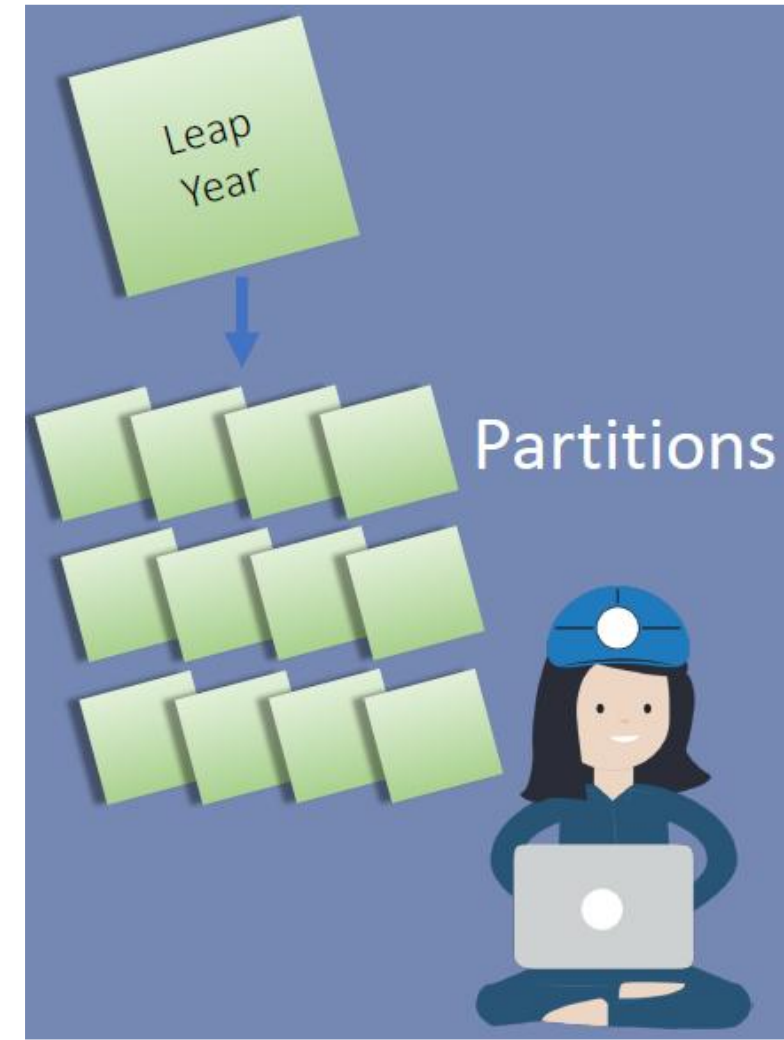


Examples on Equivalence partitioning

- So, there are three conditions
 - ❑ If we divide the year by 4, and the remainder is zero, then it is a leap year
 - ❑ However, we have some exceptions: if the year can also be divided by one hundred exactly, then it is not a leap year.
 - ❑ However, these years can still be leap, if they are divisible by four hundred.

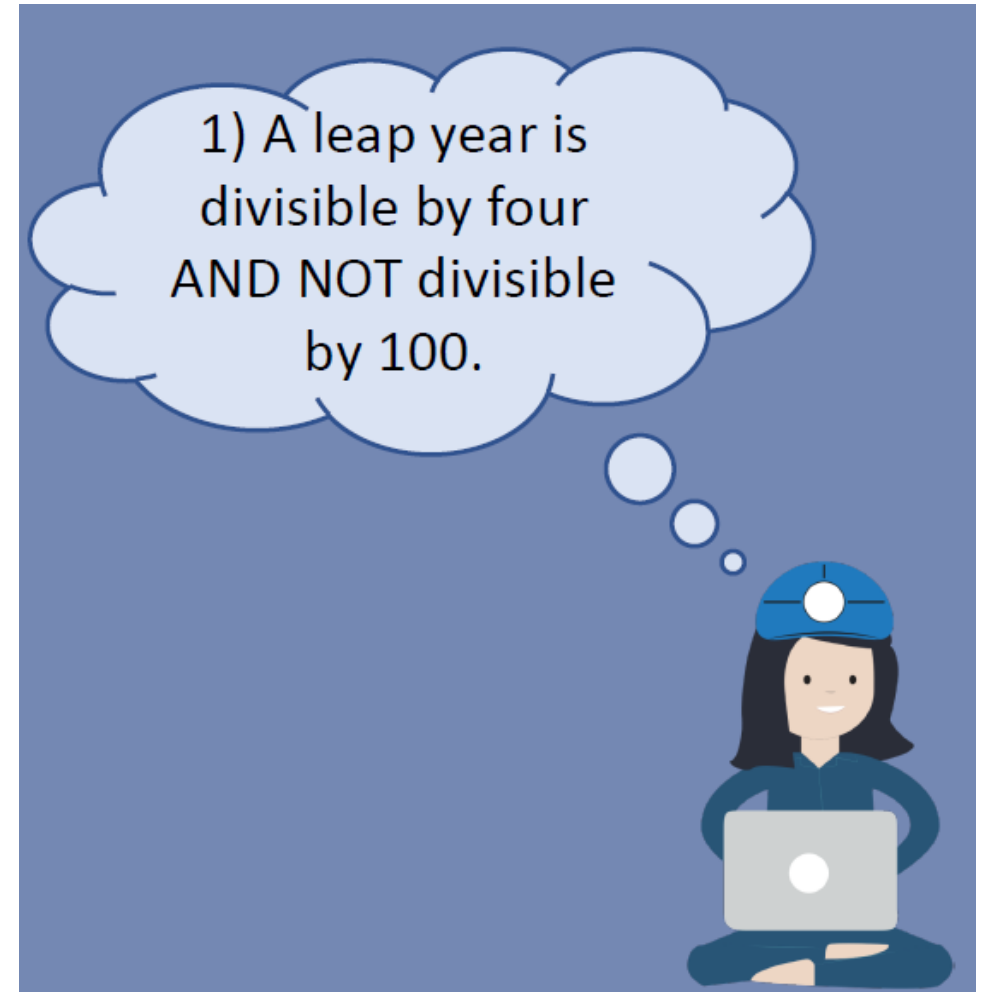
Examples on Equivalence partitioning

- So, how do we test this?
 - ❑ The first thing we should do is to break the problem into smaller partitions, or classes.
 - ❑ Each partition represents a different case that has to be tested.
 - ❑ Here, different cases represent different inputs



Examples on Equivalence partitioning

- So, Let's keep thinking about partitions



Examples on Equivalence partitioning

- So, Let's keep thinking about partitions



Examples on Equivalence partitioning

- For now, we only thought about our program identifying leap years.
- But we should also make sure that it can identify years that are ****not**** leap years.



Examples on Equivalence partitioning

- The first one is years divisible by four, and not divisible by one hundred.
- As concrete examples, years like 2016, or 2020, fall into this partition.
- Second partition is years divisible by four, divisible by one hundred, which would make this year a non- leap year.
- But if it is also divisible by four hundred, then it turns back into a leap year.
- A concrete example is the year 2000.
- Third case represents if the number is not divisible by four, the year is not a leap year.
- For example, 2015, and 2017.
- Finally, in fourth case, a year that is divisible by 4, divisible by one hundred, but not divisible by four hundred, is not a leap year.
- Examples are 1500 and 1900

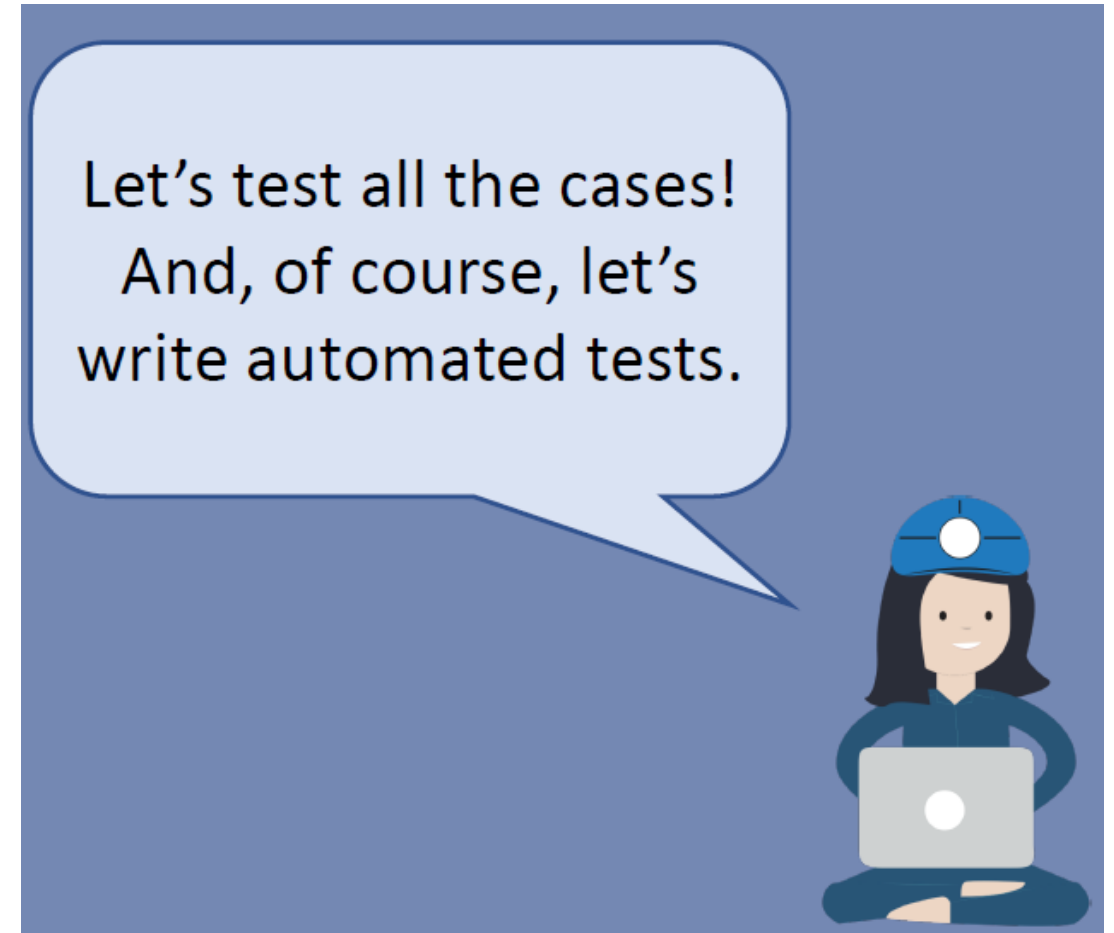
 divisible by 4,
not divisible by 100
Ex: 2016, 2020

 Divisible by 4, divisible by 100,
but also divisible by 400
Ex: 2000

 Not divisible by 4.
Ex: 2015, 2017

 Divisible by 4, divisible by 100,
but not divisible by 400
Ex: 1500, 1900

Examples on Equivalence partitioning



Examples on Equivalence partitioning

```
public class LeapYear {  
    public boolean isLeapYear(int year) {  
        if (year % 400 == 0)  
            return true;  
        if (year % 100 == 0)  
            return false;  
  
        return  
            (year % 4 == 0) ? true : false;  
    }  
}
```

Examples on Equivalence partitioning



divisible by 4,
not divisible by 100

Examples:

2016
2020
2024
2028
...

Which one should
I choose?



divisible by 4,
not divisible by 100

Any! They all belong to
the same **equivalence
partitioning!**



- Clearly, it would be impractical to test all the possible inputs for this partition.
- However, all of them are equivalent.

Examples on Equivalence partitioning

- One good tip is to actually name your test methods after the partition.

```
@Test
public void
    leapYearsThatAreNonCenturialYears() {
    LeapYear ly = new LeapYear();
    boolean leap = ly.isLeapYear(2016);
    Assertions.assertTrue(leap);
}
```

➤ 1st Partition

Examples on Equivalence partitioning

```
@Test
public void leapCenturialYears() {
    LeapYear ly = new LeapYear();

    boolean leap = ly.isLeapYear(2000);

    Assertions.assertTrue(leap);
}
```

➤ 2nd Partition

```
@Test
public void nonLeapYears() {
    LeapYear ly = new LeapYear();

    boolean leap = ly.isLeapYear(2017);

    Assertions.assertFalse(leap);
}
```

➤ 3rd Partition

Examples on Equivalence partitioning

```
@Test
public void nonLeapCenturialYears() {
    LeapYear ly = new LeapYear();

    boolean leap = ly.isLeapYear(1500);

    Assertions.assertFalse(leap);
}
```

➤ 4th Partition